

AArch64 Arithmetic and Logical Instructions

Arithmetic, logical, shift, and immediate-value instructions in AArch64 assembly

Dr. J. Burns

Topic 5 Outline

Topic Objective

Use the main AArch64 data-processing instruction families to perform arithmetic, manipulate individual bits, shift register values, and work with immediate constants.

- **Section 1:** Arithmetic instructions
- **Section 2:** Logical and bitwise instructions
- **Section 3:** Shift instructions
- **Section 4:** Immediate values and constants
- **Section 5:** Summary and self-test

Section 1

Arithmetic Instructions

Data Processing in Registers

Load/Store Architecture

AArch64 arithmetic instructions operate primarily on values already held in registers. Memory access is handled separately by load and store instructions.

```
add x0, x1, x2    sub x3, x4, x5
```

- The first operand is the destination register.
- The remaining operands provide the values used by the operation.
- The result replaces the previous contents of the destination register.
- Register operands make the data path regular and predictable.

Addition and Subtraction

Addition

`add x0, x1, x2`

- Computes $x1 + x2$.
- Stores the result in `x0`.
- Operands remain unchanged unless reused as the destination.

Subtraction

`sub x0, x1, x2`

- Computes $x1 - x2$.
- Stores the result in `x0`.
- Uses the same fixed-width register datapath.

Connection to Topic 4

Flag-setting forms such as adds and subs were covered with NZCV in Topic 4. Here the focus is instruction use and operand forms.

Register and Immediate Arithmetic

Register Operand

`add x0, x1, x2`

`sub x3, x4, x5`

- Both source values come from registers.
- Useful when both values were computed earlier.

Immediate Operand

`add x0, x1, #24`

`sub x3, x4, #8`

- One source value is encoded in the instruction.
- Useful for small fixed constants.

32-Bit and 64-Bit Arithmetic

Register Width Selects the Operation Width

The same instruction family is available for 32-bit w registers and 64-bit x registers.

`add w0, w1, w2` `add x0, x1, x2`

- w0–w30 operate on 32-bit values.
- x0–x30 operate on 64-bit values.
- Writing a w register clears the upper 32 bits of the corresponding x register.
- Operand widths must match the instruction form.

Reusing a Source as the Destination

In-Place Update

The destination register may also be one of the source registers.

```
add x0, x0, #1
```

```
sub x5, x5, x2
```

- The sources are read before the destination value is written.
- `add x0, x0, #1` increments `x0`.
- `sub x5, x5, x2` replaces `x5` with `x5 - x2`.
- This form is common in counters and address calculations.

Multiplication

Basic Multiply

```
mul x0, x1, x2
```

- Multiplies the two source registers and writes the low 64 bits to the destination.
- The 32-bit form uses w registers and writes a 32-bit result.
- Multiplication is architecturally a register data-processing operation.
- Its execution latency is a microarchitectural property, not fixed by the ISA.

Example

If $x1 = 6$ and $x2 = 7$, then `mul x0, x1, x2` leaves $x0 = 42$.

Signed and Unsigned Division

Unsigned Division

```
udiv x0, x1, x2
```

- Interprets the operands as unsigned integers.
- Quotient is truncated toward zero.

Signed Division

```
sdiv x0, x1, x2
```

- Interprets the operands as signed two's-complement integers.
- Quotient is truncated toward zero.

Why Two Instructions?

Unlike add and sub, division must know whether the operands are signed or unsigned because the quotient depends on that interpretation.

Arithmetic Instruction Summary

Instruction	Operation	Example
add	addition	add x0, x1, x2
sub	subtraction	sub x0, x1, #8
mul	multiplication	mul x0, x1, x2
udiv	unsigned division	udiv x0, x1, x2
sdiv	signed division	sdiv x0, x1, x2

Section 2

Logical and Bitwise Instructions

Bitwise Operations

Operate on Corresponding Bits

Logical instructions process each bit position independently. Signed and unsigned numeric interpretation is irrelevant to the bitwise operation itself.

`and x0, x1, x2` `orr x0, x1, x2` `eor x0, x1, x2`

- `and`: output bit is 1 only when both input bits are 1.
- `orr`: output bit is 1 when either input bit is 1.
- `eor`: output bit is 1 when the input bits differ.

AND for Selecting Bits

Masking

A bit mask uses 1s in positions that should be preserved and 0s in positions that should be cleared.

```
      11001010
    & 00111100
result 00001000
```

AArch64 Example

```
and x0, x1, x2
```

ORR for Setting Bits

Setting Selected Bits

A mask containing 1s can force selected result bits to 1 while leaving other positions unchanged.

```
      10000001
      | 00110000
result 10110001
```

AArch64 Example

```
orr x0, x1, x2
```

EOR for Toggling Bits

Toggle Selected Bits

XOR with 1 flips a bit; XOR with 0 leaves the bit unchanged.

```
      10100110
    ^ 00111100
result 10011010
```

AArch64 Example

```
eor x0, x1, x2
```

Inverted-Operand Logical Forms

Why These Instructions Exist

Some bit-mask operations need one operand inverted before AND, OR, or XOR. These forms combine the inversion with the logical operation.

Instruction	Operation	Typical use
bic	$A \wedge \sim B$	clear selected bits
orn	$A \vee \sim B$	OR with an inverted operand
eon	$A \oplus \sim B$	XOR with an inverted operand

Example: Clearing Selected Bits

```
bic x0, x1, x2
```

x2 is the mask: 1-bits in x2 clear the corresponding bits of x1; the result is written to x0.

Logical Instruction Patterns

Goal	Operation	Typical instruction
Keep selected bits	AND with 1s at wanted positions	<code>and</code>
Set selected bits	OR with 1s at wanted positions	<code>orr</code>
Toggle selected bits	XOR with 1s at wanted positions	<code>eor</code>
Clear selected bits	AND with inverted mask	<code>bic</code>

Bit Masks

Bit masks are common in device control, packed data, permission fields, status words, and instruction-field manipulation.

Section 3

Shift Instructions

Four Common Shift and Rotate Operations

Instruction	Bit movement	Typical use
lsl	left; zeros enter on right	scale by powers of two when high bits are not lost
lsr	right; zeros enter on left	unsigned right shift
asr	right; sign bit replicated	signed right shift
ror	right; outgoing bits re-enter left	rotate a bit pattern

Shift Amount

The shift amount specifies how many bit positions the value moves.

Logical Shift Left

Operation

```
lsl x0, x1, #3
```

- Moves every bit in x1 three positions toward the most-significant end.
- Zeros enter the three vacated low-order bit positions.
- Bits shifted beyond the register width are discarded.
- When no significant high bits are lost, shifting left by 3 corresponds to multiplication by 8.

Finite Width

The register does not become wider during the shift; only the destination-width bits remain.

Logical and Arithmetic Right Shift

Logical Right Shift

`lsr x0, x1, #1`

- Moves bits right by one.
- Inserts 0 into the top bit.
- Appropriate for unsigned bit patterns.

Arithmetic Right Shift

`asr x0, x1, #1`

- Moves bits right by one.
- Replicates the original sign bit.
- Preserves signed two's-complement interpretation.

Difference

For values whose top bit is 1, `lsr` inserts 0s while `asr` inserts 1s.

Rotate Right

Operation

```
ror x0, x1, #8
```

- Bits move toward lower-order positions.
- Bits leaving the right end re-enter at the left end.
- No bit positions are replaced by zeros or replicated sign bits.
- Rotation is therefore different from both logical and arithmetic shifting.

Typical Uses

Rotations appear in bit manipulation, hashing, cryptographic algorithms, and packed-data transformations.

Shifted Register Operands

Combine Shift and Operation

Many AArch64 data-processing register forms can shift one source operand as part of the same instruction.

```
add x0, x1, x2, lsl #3
```

```
orr x3, x4, x5, lsr #8
```

- The shift applies to the final source register before the main operation.
- The source register itself is not modified.
- This can combine scaling and arithmetic or bit manipulation in one architectural instruction.

Section 4

Immediate Values and Constants

What Is an Immediate?

Constant Inside the Instruction

An immediate operand is a constant represented within the instruction itself rather than read from a source register.

```
add x0, x1, #5
```

```
sub x2, x2, #32
```

- The # prefix identifies an immediate constant in AArch64 assembly syntax.
- Immediate forms are convenient for fixed constants known when the program is assembled.
- The constant must fit the encoding permitted by that particular instruction form.

Immediate Encodings Depend on the Instruction

No Universal Immediate Format

AArch64 does not have one common immediate field that works identically for every instruction class.

- add and sub use an arithmetic-immediate form designed for common small constants.
- Logical instructions use a specialized bit-mask immediate representation.
- Shift instructions encode the shift amount according to their operation width.
- Move-wide instructions place 16-bit pieces into selected portions of a register.

Encoding Boundary

The exact bit fields are studied later in Topic 9. Here the important point is that immediate availability is instruction-specific.

Logical Immediate Masks

Useful but Restricted

Logical-immediate forms can encode many repeated bit-mask patterns directly, but not every arbitrary 32-bit or 64-bit constant.

```
and x0, x1, #0xff
orr x2, x2, #0x100
```

- These forms are especially useful for masking and setting fields.
- Whether a particular mask is encodable is determined by the logical-immediate encoding rules.
- If it is not encodable directly, the constant can first be constructed in a register.

Building Constants with `movz` and `movk`

`movz`

Moves a 16-bit immediate into a selected 16-bit position and sets the remaining register bits to zero. `movz x0, #0x1234`

`movk`

Moves a 16-bit immediate into a selected 16-bit position while keeping the other bits unchanged. `movk x0, #0xabcd, lsl #16`

Purpose

Several move-wide instructions can assemble a full 64-bit constant from 16-bit pieces without loading it from memory.

movn and mov

movn

- Moves the bitwise NOT of a 16-bit immediate into a selected position.
- Other bits are set consistently with the move-wide operation.
- Can efficiently construct constants containing many 1s.

mov

- Common assembler syntax for putting a value into a register.
- Depending on the operands, the assembler selects an appropriate underlying instruction form.
- It should not be assumed to correspond to one universal hardware encoding.

```
mov x0, x1    mov x2, #42
```

Constant Construction Example

Construct 0x123456789abcdef0

A 64-bit constant can be built as four 16-bit pieces.

```
movz x0, #0xdef0
movk x0, #0x9abc, lsl #16
movk x0, #0x5678, lsl #32
movk x0, #0x1234, lsl #48
```

Result

After the sequence, x0 contains 0x123456789abcdef0.

Choosing Between an Immediate and a Register

Use an Immediate When

- the constant is known at assembly time,
- it fits the instruction's immediate form, and
- no register value is required for reuse elsewhere.

Use a Register When

- the value is computed at runtime,
- the constant does not fit the immediate form, or
- the value will be reused by several instructions.

Compiler Role

Compilers and assemblers choose among these forms to express the required operation using legal A64 encodings.

Combined Data-Processing Example

Goal

Scale a value by 8, add it to a base value, then keep only the low byte.

```
add x0, x1, x2, lsl #3
and x0, x0, #0xff
```

- The first instruction shifts `x2` left by 3 and adds it to `x1`.
- The second instruction masks the result to retain only bits 7–0.
- Both operations remain entirely within the register data-processing model.

Section 5

Summary and Self-Test

Topic 5 Summary

- `add`, `sub`, `mul`, `sdiv`, and `udiv` provide the main arithmetic operations introduced here.
- `and`, `orr`, `eor`, and `bic` manipulate selected bit positions.
- `lsl`, `lsr`, `asr`, and `ror` rearrange bits in different ways.
- Many AArch64 data-processing instructions can combine a shifted register operand with the main operation.
- Immediate constants are encoded according to instruction-specific rules.
- `movz`, `movk`, and `movn` help construct constants that do not fit a simple immediate form.

Self-Test 1/2

- ① What is the difference between `add x0, x1, x2` and `add x0, x1, #8`?
- ② What happens to the upper 32 bits of `x0` after writing `w0`?
- ③ Why are separate `sdiv` and `udiv` instructions required?
- ④ Which logical instruction is useful for keeping only selected bits?
- ⑤ How can `eor` be used to toggle selected bits?

Self-Test 2/2

- 6 What is the difference between `lsr` and `asr`?
- 7 Why is `ror` not equivalent to a right shift?
- 8 What does the shifted operand mean in `add x0, x1, x2, lsl #3`?
- 9 Why can an immediate operand not represent every possible 64-bit constant?
- 10 What roles do `movz` and `movk` play when constructing a 64-bit constant?

Data Transfer and Addressing

Topic 6 moves from register data processing to the AArch64 instructions that transfer values between registers and memory.

- Loads and stores
- Transfer widths and extension
- Base-plus-offset addressing
- Pre-indexed and post-indexed addressing
- Arrays, pointers, and structured memory access